INTERMEDIATE SERIES · GUIDE 4 OF 8

Intermediate Agentic AI

LangGraph stateful agents, human-in-loop, multi-agent orchestration, agent evaluation

01 LangGraph — Stateful Agent Graphs

LangGraph represents agent workflows as directed graphs where nodes are functions and edges define transitions. Unlike simple chains, graphs can have **cycles** (loops) and **conditional branches** — essential for real agents.

```
# LangGraph — ReAct agent with state
from langgraph.graph import StateGraph, END
from langgraph.prebuilt import ToolNode
from langchain_openai import ChatOpenAI
from typing import TypedDict, Annotated, List
import operator

# Define agent state
class AgentState(TypedDict):
    messages: Annotated[List, operator.add]  # append-only list

# Nodes
def agent_node(state: AgentState):
    """LLM decides what to do next."""
    llm = ChatOpenAI(model="gpt-4o-mini")
    llm_with_tools = llm.bind_tools(tools)
    response = llm_with_tools.invoke(state["messages"])
    return {"messages": [response]}

tool_node = ToolNode(tools)  # executes tool calls

# Conditional edge: route to tools or end
def should_continue(state: AgentState):
    last = state["messages"][-1]
    if last.tool_calls:
        return "tools"  # has tool calls → execute them
    return END  # no tool calls → done

# Build graph
graph = StateGraph(AgentState)
```

```
graph.add_node("agent", agent_node)
graph.add_node("tools", tool_node)
graph.set_entry_point("agent")
graph.add_conditional_edges("agent", should_continue)
graph.add_edge("tools", "agent")  # after tools → back to agent
app = graph.compile()

# Run
result = app.invoke({"messages": [HumanMessage(content="Search for AI news")
]})
```

02 Human-in-the-Loop Patterns

Production agents often need to pause for human approval before taking irreversible actions — sending emails, making payments, deleting data. LangGraph supports this natively via checkpointing and interrupts.

```
# Human approval interrupt before action
from langgraph.checkpoint.memory import MemorySaver
from langgraph.graph import StateGraph, END

checkpointer = MemorySaver()  # persist state between interactions

def review_action(state):
    """Pause here and wait for human approval."""
    # This node returns but graph is INTERRUPTED
    return state  # state is saved by checkpointer

graph = StateGraph(AgentState)
graph.add_node("plan_action", plan_action)
graph.add_node("human_review", review_action)
graph.add_node("execute", execute_action)
graph.add_edge("plan_action", "human_review")
graph.add_edge("human_review", "execute")
app = graph.compile(
    checkpointer=checkpointer,
    interrupt_before=["execute"]  # PAUSE before execute node
)

# First run — pauses before execute
thread = {"configurable": {"thread_id": "task_1"}}
result = app.invoke({"messages": [HumanMessage("Send email to team")]}, thre
ad)

# Human reviews...
print("Pending action:", app.get_state(thread).values["messages"][-1])

# Human approves — resume
app.invoke(None, thread)  # None = continue from checkpoint
```

03 Multi-Agent Orchestration with LangGraph

Complex tasks can be broken across multiple specialist agents. A supervisor agent routes tasks; worker agents execute them.

```
# Supervisor + worker agent pattern
from langchain_core.messages import HumanMessage
from langchain_openai import ChatOpenAI

# Define worker agents
researcher = create_agent(llm, [search_tool], "Research specialist")
coder      = create_agent(llm, [python_tool], "Python coding specialist")
analyst    = create_agent(llm, [calc_tool],   "Data analysis specialist")

workers = {"researcher": researcher, "coder": coder, "analyst": analyst}

# Supervisor: decides which worker to call
SUPERVISOR_PROMPT = """
Given the task and conversation, decide who to call next.
Workers: researcher, coder, analyst, FINISH
Respond with: {"next": "worker_name"}
"""

def supervisor_node(state):
    response = llm.invoke([SystemMessage(SUPERVISOR_PROMPT)]
                          + state["messages"])
    next_worker = json.loads(response.content)["next"]
    return {"next": next_worker}

# Route to workers or finish
def route(state):
    return END if state["next"] == "FINISH" else state["next"]

graph = StateGraph(AgentState)
graph.add_node("supervisor", supervisor_node)
for name, agent in workers.items():
    graph.add_node(name, agent)
    graph.add_edge(name, "supervisor")  # all workers report back
graph.add_conditional_edges("supervisor", route)
graph.set_entry_point("supervisor")
app = graph.compile()
```

04 Agent Evaluation and Reliability

Measuring Agent Quality

Metric	What It Measures	How to Measure

Task Completion Rate	Does agent successfully finish tasks?	Run on test set, check if final output is correct
Tool Call Accuracy	Does agent call the right tools with right args?	Compare tool_call logs vs expected tool sequence
Step Efficiency	Does agent use minimum steps to complete task?	Count LLM calls per task — fewer is better
Error Recovery Rate	When a tool fails, does agent recover?	Inject tool errors in tests, check if agent retries/adapts
Hallucination Rate	Does agent invent tool results or capabilities?	Check tool call args vs actual available tool schemas

Agent Reliability Patterns

Problem	Pattern	Implementation
Tool call fails	Retry with backoff	Wrap tool node with try/except + re-add error to messages
Agent loops infinitely	Max iterations limit	Count steps in state, END if steps > max_steps
Agent takes wrong action	Human-in-loop approval	interrupt_before on sensitive nodes
LLM returns invalid JSON for tool	Output validation + retry	Pydantic validation of tool args, re-prompt if invalid
Agent cost explodes	Token budget tracking	Track tokens in state, switch to cheaper model after threshold

```
# Max iterations safety guard
class AgentState(TypedDict):
    messages: Annotated[List, operator.add]
    steps: int  # track iterations

def agent_node(state):
    if state["steps"] >= 10:  # safety limit
        return {"messages": [AIMessage("Max steps reached.")],
                "steps": state["steps"]}
    response = llm_with_tools.invoke(state["messages"])
    return {"messages": [response], "steps": state["steps"] + 1}
```